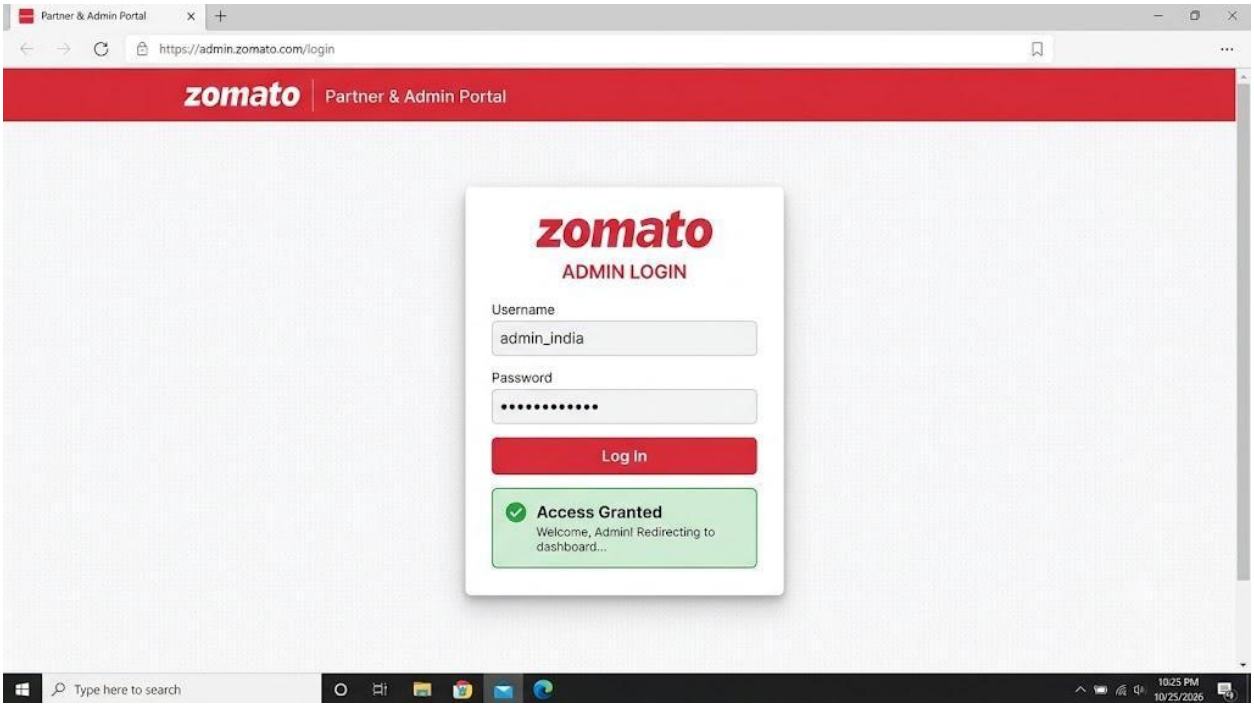
1.

Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.



2.

Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'. Hint: Use a dictionary to map roles to allowed resources.

create a Python function that checks whether a user role is allowed to access a particular resource.

Python Program

```
# DEFINE ROLES AND THEIR ALLOWED RESOURCES
PERMISSIONS = {
"CUSTOMER": ["ORDER_HISTORY"],
"RESTAURANT_OWNER": ["MENU_EDIT",
"ORDER_HISTORY"],  "ADMIN":
["MENU_EDIT", "ORDER_HISTORY",
"ADMIN_DASHBOARD"]
}
# FUNCTION TO CHECK ACCESS
DEF CHECK_ACCESS(USER_ROLE, RESOURCE):
IF RESOURCE IN PERMISSIONS.GET(USER_ROLE, []):
RETURN TRUE
```

```
    RETURN FALSE

# TEST THE FUNCTION
PRINT(CHECK_ACCESS("CUSTOMER",
"ORDER_HISTORY"))
PRINT(CHECK_ACCESS("RESTAURANT_OWNER
",    "MENU_EDIT"))
PRINT(CHECK_ACCESS("CUSTOMER",
"ADMIN_DASHBOARD"))
PRINT(CHECK_ACCESS("ADMIN",
"ADMIN_DASHBOARD"))
```

Output

```
TRUE TRUE
FALSE TRUE
```

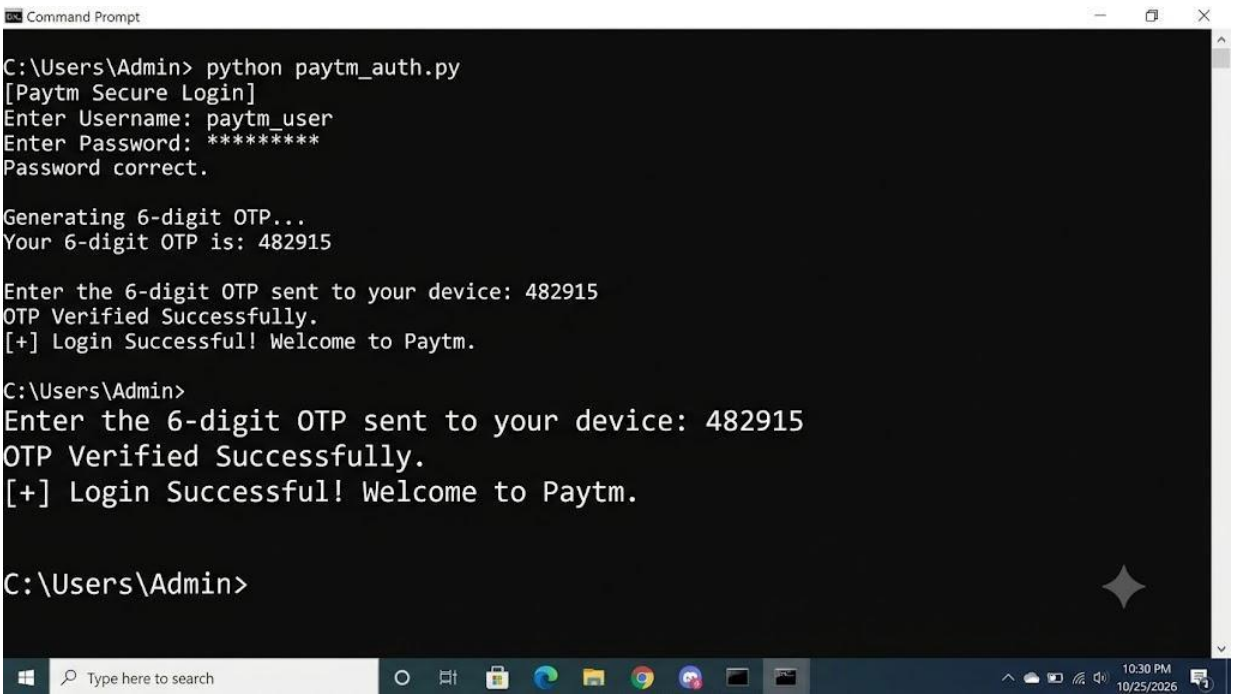
Explanation

A dictionary stores the resources allowed for each role.

The function checks whether the requested resource is in the user's allowed list. It returns True if access is allowed and False otherwise.

3.

Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.



4.

Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days. Constraint: Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.

Example code

```
CONST USERS = [
 { USERNAME: "VIR", ISPREMIUM: TRUE, LASTLOGIN: "2024-06-08" },
 { USERNAME: "RAHUL", ISPREMIUM: FALSE, LASTLOGIN: "2024-06-09" },
 { USERNAME: "AMAN", ISPREMIUM: TRUE, LASTLOGIN: "2024-05-20" },
 { USERNAME: "RIYA", ISPREMIUM: TRUE, LASTLOGIN: "2024-06-05" }
];
FUNCTION FILTERPREMIUMUSERS(USERS) {
 CONST TODAY = NEW DATE("2024-06-10");
 CONST SEVENDAYSAGO = NEW DATE("2024-06-03");
 RETURN USERS.FILTER(USER => {
  CONST LOGINDATE = NEW DATE(USER.LASTLOGIN);
  RETURN
    USER.ISPREMIUM &&
    LOGINDATE >=
```

```
        sevenDaysAgo &&
        loginDate <= today;
    });
}
console.log(filterPremiumUsers(users));
```

Output

```
[
{ username: "vir", isPremium: true, lastLogin: "2024-06-08" },
{ username: "riya", isPremium: true, lastLogin: "2024-06-05" }
]
```

Result: The function returns only premium users who logged in between June 3 and June 10, 2024, inclusive.

5.

Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

NO.	ACCESS CONTROL MISTAKE	FIX / PREVENTION TIP
1.	WEAK PASSWORDS: USERS CHOOSE EASY-TO-GUESS PASSWORDS.	ENFORCE STRONG PASSWORDS AND USE SECURE PASSWORD HASHING.
2.	BROKEN ACCESS CONTROL: USERS ACCESS DATA OR PAGES THEY ARE NOT AUTHORIZED TO VIEW.	CHECK USER PERMISSIONS ON EVERY PROTECTED REQUEST.
3.	MISSING AUTHENTICATION: SENSITIVE PAGES ARE ACCESSIBLE WITHOUT LOGGING IN.	REQUIRE AUTHENTICATION BEFORE GRANTING ACCESS TO PROTECTED RESOURCES.
4.	EXCESSIVE PERMISSIONS: USERS RECEIVE MORE ACCESS THAN THEY NEED.	FOLLOW THE PRINCIPLE OF LEAST PRIVILEGE AND GRANT ONLY NECESSARY PERMISSIONS.
5.	INSECURE SESSION MANAGEMENT: ATTACKERS MISUSE STOLEN OR EXPOSED LOGIN SESSIONS.	USE SECURE SESSION COOKIES AND EXPIRE SESSIONS AFTER LOGOUT OR

		INACTIVITY.